

UnifiedCode — Real-Time Collaborative Code Editor and Whiteboard

Yash Kumar Aggarwal^{1†}, Dishant Rajput^{1†},
Vijay Kumar Chaurasiya^{1†}, Vishnu Kumar Verma^{1†},
Preeti Garg^{1*}

^{1*}Computer Science And Engineering, KIET group of Institutions,
Ghaziabad, 201206, Uttar Pradesh, India.

*Corresponding author(s). E-mail(s): preeti.itgarg@gmail.com;
Contributing authors: yashaggarwal2002.ya@gmail.com;
rajputdishant891@gmail.com; vijaykumarchaurasiya999@gmail.com ;
vishnukumarverma574@gmail.com ;

[†]These authors contributed equally to this work.

Abstract

Modern development includes distributed, disparate tools for editing code, messaging, and diagramming, constant concurrency between editors, chat apps, and whiteboards. UnifiedCode provides a single browser-based workspace that integrates a real-time multi-user code editor, synchronized file management, chat, and a collaborative whiteboard. The system uses React frontend (CodeMirror 6 for code editing, tldraw for sketching) and an Express, Socket.IO backend. Collaboration actions are encoded as room-scoped Socket.IO events.

Code execution is offered through the Piston API to avoid bundling runtimes. Testing on typical desktop hardware showed low event latencies and consistent shared state. The design favors simplicity over full complexity. Future work includes hybrid OT/CRDT concurrency handling, persistence/versioning, role-based access control, and scalability evaluation under adverse network conditions.

Keywords: Real-Time Collaboration, Socket.IO, Collaborative Editing, React, CodeMirror, Express.js, Systems Architecture

1 Introduction

Modern software development increasingly demands real-time collaboration, yet developers rely on different tools, separate applications for code editing, messaging, video calls, and diagramming etc.. This creates overhead through constant context switching, particularly during pair programming, technical interviews, and remote teamwork.

Existing solutions include cloud IDEs that require heavy installations with limited communication features with browser platforms that treat collaboration as secondary.t. Meanwhile, collaborative document editors like Google Docs prove that seamless multi-user editing is possible but don't address the specialized needs of software development. UnifiedCode consolidates four critical capabilities into a single browser-based workspace: real-time multi-user code editing with syntax highlighting and cursor awareness across multiple files, integrated text chat for seamless communication within the coding context, collaborative vector whiteboard for visualizing algorithms and system architectures, and multi-language code execution supporting 40+ languages through secure external APIs. The project uses React.js for the frontend, CodeMirror 6 for editing, tldraw for vector drawing, and Node.js with Socket.IO for real-time communication.

Rather than complex algorithms, UnifiedCode employs server-sequenced events—every action receives a canonical sequence number from the server, preventing state divergence while maintaining performance for typical team sizes. The architecture uses four guiding principles—simplicity, modularity, performance, and openness- that shaped every design decision. This paper presents UnifiedCode's complete development and evaluation across six sections, covering system architecture, implementation methodology, experimental results, and future research directions.

2 Literature Survey

The development of collaborative software engineering tools reflects decades of research in distributed collaborative systems and computer-computer interaction. Understanding this evolution is essential for UnifiedCode's design, and it also highlights remaining gaps that motivate continued innovation. The theoretical foundations of real-time collaborative editing emerged in the late 1980s as researchers explored how multiple users could simultaneously modify shared documents without conflicts. Operational Transformation (OT), introduced by Ellis and Gibbs in the GROVE system, provided the first practical solution by algorithmically transforming concurrent changes so all users eventually converge to the same final state. Sun et al. formalized these transformation functions and proved conditions guaranteeing convergence and consistency. OT became the foundation for commercial collaborative editors including Google Docs, though its implementation complexity has challenged developers for decades.

An alternative approach emerged in the 2000s with Conflictfree Replicated Data

Types (CRDTs), which guarantee eventual consistency through mathematically-proven commutative operations that can be applied in any order without coordination.

Collaborative editing of documents was a solution to core synchronization problems; however, applying it to source code needs. Source code is different than prose in that it has a syntactic structure that must be preserved. Also language-specific features, like autocompletion and refactoring, require that accurate abstract syntax trees be maintained in sync, side by side with the text.

CodeMirror has emerged as one of the most prominent code editor frameworks for the browser with syntax highlighting, smart indenting, and a plugin architecture. The transaction-based update model naturally supports collaborative editing. At the time of writing, Visual Studio Code Live Share was also already showing enterprise-grade collaborative coding, integrated in a full-featured IDE. Developers can share not just their coding project, but the entire workspace, including terminals, debuggers, etc. But, this richness must be installed, works through Microsoft cloud infrastructure and can feel heavyweight for quick sessions. The cloud IDEs developed by browser-based platforms like Replit and CodeSandbox run only on browsers. However, the collaborative features in these platforms feel more like secondary features in IDEs with a full development environment.

CodderPad and HackerRank are examples of the second type —which are optimized for technical interviews but not flexible for open-ended collaborative development. The technical infrastructure enabling real-time web applications underwent an unprecedented change with the establishment of the WebSocket protocol standard in 2011, allowing for the full-duplex communication. Socket.IO was developed on top of WebSockets, adding important features for production like: reconnecting automatically, acknowledging messages, routing based on namespaces, and using fallback when WebSockets are blocked. It has a room-based architecture that maps naturally to collaborative sessions. Users join named channels. A lot of issues arise when any user can run code safely. It is very hard to let users run any code safely. Modern code execution APIs like Judge0 and Piston utilise containerization to execute the code in isolated Docker containers which has strict resource limit. These services deal with the complicated infrastructure that manages specific language runtimes, which also ensure that malicious code does not affect other users or the host itself.

3 System Architecture

UnifiedCode embraces a clear separation between the presentation client-side and the coordination on the serverside, which are connected through a standardized event-driven protocol. This separation allows for independent evolution of the frontend and backend while maintaining a stable contract between the two. The system has three main architectural layers: the client application (browser-based frontend), the coordination server (backend event hub), and external services (code execution API).

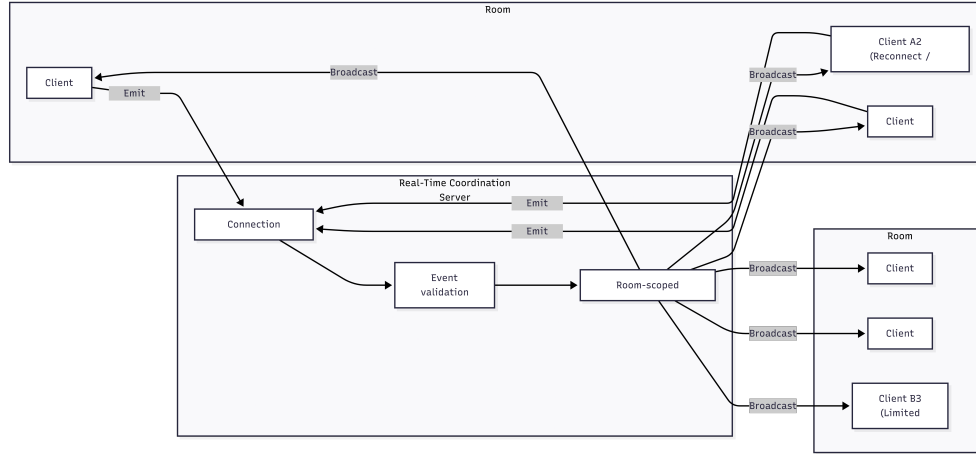


Fig. 1 Flow Diagram

3.1 Architectural Overview

The fundamental architectural pattern is client-server with event-driven synchronization. Unlike traditional request-response web applications, where clients explicitly request data and wait for responses, UnifiedCode employs bidirectional event streams where both clients and server asynchronously emit events representing state changes. Visual communication is an important aspect of collaborative programming when explaining algorithms or sketching architectures. Traditional whiteboard applications such as Miro are stand-alone platforms that require context switches. tldraw, launched in 2021, is part of a new generation of embeddable whiteboard components targeted for integration into web applications. An event-based API emits fine-grained drawing operations that can be transmitted across network connections and replayed on remote clients.

Despite decades of improvements, state-of-the-art platforms have persistent limitations: fragmentation of features in which users need to keep several tools for different purposes; complexity barriers where installation and configuration are required; inaccessibility, where important features are tied to expensive licenses; heavyweight architectures that load a fullfledged development environment for simple tasks; lack of customization possibilities; and neglect of education through oversimplification or loss of features. This event-centric design naturally models the collaborative interactions: when a user types a character, creates a file, or draws on the whiteboard, these actions become events that flow to the server and broadcast to other participants. Event-driven architecture offers several advantages for realtime collaboration:

- **Loose Coupling:** Components communicate through welldefined events rather than direct method calls, allowing independent implementation and testing.

- **Natural Concurrency Model:** Events represent actions in temporal order, making concurrent interactions easier to reason about than shared mutable state.
- **Extensibility:** New features can be added by defining new event types and handlers without modifying existing code paths.
- **Observable Behavior:** Event logs provide complete records of collaboration sessions, useful for debugging, auditing, and replay.

3.2 Client Layer Architecture

The client application runs entirely within modern web browsers, requiring no installation or plugins beyond standard HTML5/ES6 support. Built with React 18 and TypeScript, the frontend employs a component-based architecture where each UI element (editor, file tree, chat panel, whiteboard) exists as an independent React component with clearly defined props and state.

3.2.1 Component Architecture

The client employs a hierarchical component structure: **App Component:** The root component establishes global state (user identity, WebSocket connection, current room) using React Context API. Child components access this global state without explicit prop drilling.

- **Room Component:** handles the main collaborative working area, which is divided into resizable panels. Handles room joining, user list management, and coordinates between subcomponents.
- **Editor Component:** Wraps CodeMirror 6 and manages file content, syntax highlighting, and cursor synchronization.
- Emits edit events when local content changes; applies remote edit events as CodeMirror transactions.
- **FileTree Component:** Displays the file tree with icons and is capable of performing CRUD operations on files/folders. It also tracks the currently selected file.
- **Chat Component:** It displays the message history, handles message input, and provides typing indicators. Messages are to be time-stamped, pulled for specific users.
- **Whiteboard Component:** Embeds tldraw canvas, performs event-based synchronization of drawing operations. Provides a toolbar for selecting shapes and drawing mode.
- **Console Component:** Provides the result of code execution including stdout, stderr, compilation error, and execution statistics: runtime, memory usage, exit code.

3.2.2 CodeMirror 6 Integration

CodeMirror 6 is a complete rewrite of this popular browserbased code editor with a focus on modularity, accessibility, and performance [7]. Instead of using one monolithic editor, CodeMirror 6 consists of a set of independent packages that can be combined into an editor fitting a particular need. Key features utilized in UnifiedCode include:

- **Language Support:** CodeMirror 6 has syntax highlighting for dozens of programming languages included via language packages (@codemirror/lang-javascript, @codemirror/langpython, etc.). These bundles contain parsers that create the abstract syntax trees that are used by intelligent indentation and code folding.
- **Transaction System:** All changes to the editor state, whether from user input or remote synchronization, are expressed as transactions themselves. This uniform model simplifies conflict resolution and enables undo/redo functionality.
- **Extension System:** The facet and extension mechanism of CodeMirror 6 allows custom behaviors at various points in the editor lifecycle to be injected.
- **Accessibility:** UnifiedCode provides accessibility because of its built-in ARIA annotations and keyboard navigation, which guarantee that the editor is compatible with screen readers and keyboard-only interaction.

3.3 Server Layer Architecture

The primary role of the server is that of event sequencing and broadcasting: to impose a canonical ordering on events and to distribute them to the interested clients. This automates, through centralization, the ordering ambiguities that complicate peer-to-peer collaboration.

3.3.1 tldraw Integration

Unlike raster-based canvas drawing, tldraw uses SVG for crisp rendering at any zoom level and produces structured shape data, i.e, rectangles, ellipses, arrows, and texts. This structural representation enables efficient synchronization by transmitting only shape operations, i.e., create, move, resize, and delete, rather than full image data. tldraw's event system emits fine-grained updates, which UnifiedCode packages into events and broadcasts through Socket.IO. Remote drawing events are applied through tldraw's imperative API, which batch-updates shapes to maintain smooth rendering performance

3.3.2 Socket.IO Client Integration

The socket.io-client library handles a WebSocket connection and events. On mounting the application, the client connects to the server, then joins a room (identified by URL token), and finally registers event handlers for several collaboration events:

Room Isolation: Socket.IO rooms provide logical channels. Users joining the same room receive each other's events, while different rooms remain completely isolated. This architecture allows a single server to support many concurrent collaboration sessions

1) Technology Stack

Node.js provides the runtime environment for JavaScript. Node is particularly well-suited for applications like UnifiedCode, where most time is spent waiting for network I/O- receiving events from clients and broadcasting to other clients-rather than performing CPU-intensive computation, due to its event-driven, non-blocking I/O model. A single Node.js process is capable of handling thousands of concurrent WebSocket connections using minimal memory. Express.js handles the HTTP concerns: serving the static frontend bundle, providing health check endpoints, and can expose

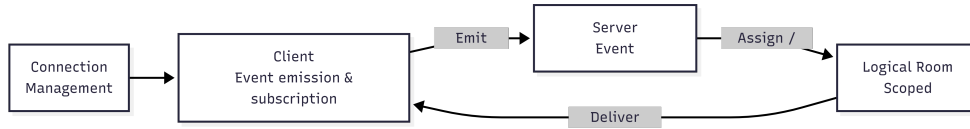


Fig. 2 Socket Connection Management

REST APIs for future features such as user authentication or session persistence. Express’s middleware architecture enables concerns such as logging, error handling, and CORS to be composed independently. Socket.IO Server handles WebSocket connections and rooms. The server-side instance of Socket.IO mirrors the client-side API. `io.on('connection' , callback)` is used for handling new connections while per-socket `socket.join(roomId)` assigns sockets to rooms. `io.to(roomId).emit(event, data)` broadcasts events to all sockets in a particular room.

2) State Management

The server maintains minimal ephemeral state:

- **Room Registry:** Maps room IDs to sets of connected sockets, used for targeted broadcasting.
- **User Registry:** Maps sockets to user metadata, such as username, connection time, and last activity, for presence tracking.
- **Active File State:** For each room, stores the current content of files in memory to support late-joining users who need to synchronize initial state.
- **Chat History:** Stores the most recent messages, such as the last 100, to allow new joiners to view the conversation context.

Notice that this server does not maintain any state across restarts: when the server restarts, all rooms are cleared. This limits availability somewhat, but makes implementation much easier, and avoids difficult questions about data ownership, retention, and privacy.

4 Proposed Methodology

The methodology involves developing a wide range of subsystems that was necessary for the UnifiedCode implementation, including integrated communication, file management, collaborative code editing, real-time synchronization infrastructure, and whiteboard features. The design and implementation process for each component is explained in the sections that follow.

4.1 Real-Time Event Synchronization

This synchronization model prioritizes consistency and simplicity for all features that ensure collaboration.

4.1.1 WebSocket Connection Management

Each client establishes a persistent WebSocket connection with the following configuration:

- WebSocket transport preferred with long polling as a fallback for restrictive networks
- Infinite automatic reconnection is essential for mobile users with intermittent connectivity
- Exponential backoff retry delays increasing from 1s to 5s maximum to avoid server overload
- React hooks manage connection lifecycle, status updates, and state synchronization after reconnection

4.1.2 Room-Based Isolation

Clients join URL-embedded rooms upon connection, creating logical channels that ensure:

- **Isolation:** Users in different rooms never receive each other's events
- **Targeted Broadcasting:** Events broadcast only to relevant room participants
- **State Synchronization:** New joiners immediately receive the current room state
- **Presence Awareness:** All participants know who else is in the room

4.1.3 Event Emission Pattern

All collaborative actions follow a consistent pattern:

- Client applies changes locally immediately (optimistic update)
- Client emits an event to the server with change details and metadata
- The server validates and broadcasts to all other room participants
- Remote clients receive and apply changes, ignoring their own echoed events
- Server adds sequence numbers to track consistency

4.1.4 Conflict Resolution

- **Concurrent file operations:** The server handles requests in the order they arrive, and the second request fails politely with an error message.
- **Fast sequential edits:** The CodeMirror transaction system automatically merges changes using offset-based positioning at high speed.
- **Network partitions:** The client detects state divergence via sequence numbers and requests full state synchronization using the "last write wins" approach.

4.2 Collaborative Code Editing

1) Editor Initialization:

Each file gets a CodeMirror 6 instance with preconfiguration:

- Language-specific syntax highlighting

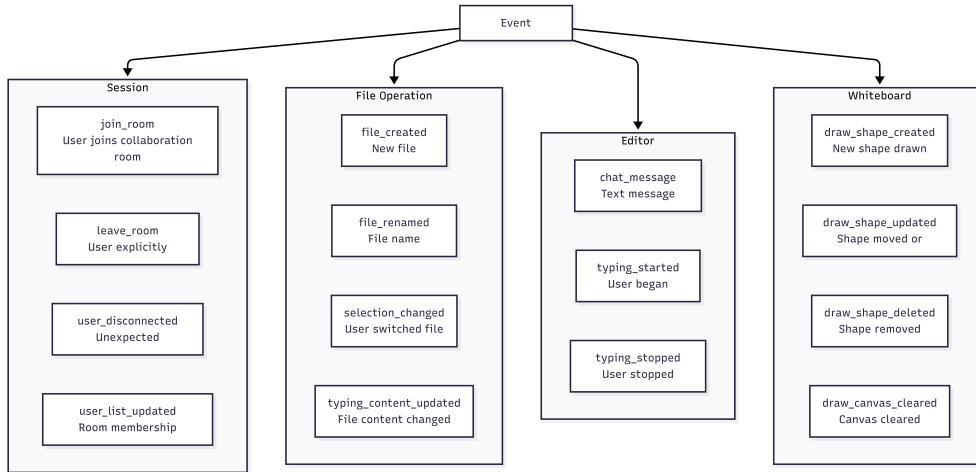


Fig. 3 Event Taxonomy Diagram

- Update listeners for local edit detection
- Remote cursor extension displaying the locations of collaborators
- Support for custom themes and keymaps

2) Local Edit Detection:

The CodeMirror's update listeners fire on every content change, capturing position, inserted text, deleted text, and cursor location, then it broadcasts it to the server.

3) Remote Edit Application:

All the incoming edits from other users are applied as per CodeMirror transactions asap.

4) Remote Cursor Indicators:

The Visual awareness is achieved through:

- Each user is assigned a unique color within the color theme itself.
- Cursor widgets display the username in a colored flag at the cursor position
- Positions automatically adjust as the document changes

5) Language Support :

40+ programming languages supported with auto-detection from file extensions and realtime syntax highlighting

4.3 File Management System

1) Tree Structure Files

Organized in a hierarchical tree with nodes containing:

- Unique ID, name, and type (file/folder)
- Content (files only) and children (folders only)
- Parent ID and auto-detected language

2) File Operations

- **Create:** Generate a new file with a unique ID, update the local tree, broadcast to others
- **Rename:** Validate no conflicts, update local state, broadcast changes
- **Delete:** Confirm if unsaved changes exist, close active editor if needed, remove from tree, broadcast deletion.

3) File Selection:

Clicking files switches the active editor by:

- Saving current file state
- Loading new file editor
- Notifying collaborators (presence indicators show colored dots next to files others are viewing)

4) Upload/Download

- **Upload:** FileReader API imports local files into the editor
- **Download:** Creates Blob objects for individual file download or JSZip for entire project download as ZIP

4.4 Multi-Language Code Compilation

1) Execution Request Flow

- Frontend initiates execution with language, code, and optional stdin
- Backend proxies request to the Piston API with appropriate language configuration
- Piston executes code in an isolated environment
- Results (output, errors, runtime, memory, exit code) returned to the frontend
- Output displayed in the console panel

2) **Language Detection File extensions** automatically map to the appropriate language/compiler from 40+ supported languages

3) Output Display Console panel shows:

- Standard output and error messages
- Syntax highlighting for error patterns
- Execution statistics (runtime in ms, memory in KB)
- Exit code and signals

4) Stdin Support:

A Dedicated textarea allows users to provide standard input for programs requiring interactive input

4.5 Integrated Text Chat

1) Message Structure:

- Messages contain ID, user ID, username, content, timestamp, and type (regular message or system notification)

2) Message Flow

- User types and sends a message
- Message added to local chat immediately
- Broadcast to all other room participants
- Remote users receive the message, and add to history
- Notifications are shown if the chat panel is hidden, and the sound plays

3) Typing Indicators

- Emitted when the user begins typing
- Cleared after 1 second of inactivity
- Displayed as animated dots with typing user names

4.6 User Interface Customization

1) System of Themes Supported color themes include:

- Solarized, monokai, dracula, light, and dark
- updates all of the interface's CSS variables.
- changes the CodeMirror theme settings that are stored in
- localStorage

2) Font Choice 16+ coding-optimized monospace fonts:

- Consolas, JetBrains Mono, Source Code Pro, and Fira Code
- Size adjustment (10-24px)
- Ligature support for operators is optional.
- Local preferences remained

3) Customization of Layout Adaptable workspace configuration:

- Panels that can be resized (file tree, editor, console/chat, whiteboard)
- Usable layouts are prevented by minimum size restrictions.
- Switch the visibility of the panel.
- Auxiliary panel tabbed interface
- Across sessions, the layout is saved and restored.

5 Results and discussions

A comprehensive evaluation of UnifiedCode assessed multiple dimensions, i.e, real-time synchronization performance, resource utilization, consistency guarantees, and user experience. The Testing was conducted on typical desktop hardware to establish baseline performance characteristics as per pseudoreal-time conditions.

5.1 Real-Time Synchronization Latency

The latency measures the time from when a user performs an action till the other users observe that action reflected in their interface.

Table 1 Latency Comparison Table

Event Type	2 Users (ms)	5 Users (ms)	10 Users (ms)
Code Edit	50–80	120–150	180–250
Cursor Movement	40–70	100–140	160–220
Chat Message	15–30	40–60	70–100
Drawing Update	60–90	150–220	250–350
File Operation	45–75	110–160	170–240

The latency results shows that UnifiedCode maintains acceptable real-time performance for all scenarios. For the primary use case (2-5 concurrent users), latencies remain below 150ms for most operations, a latency imperceptible to users whose reaction time averages 200- 300ms [1]. For under stress conditions with 10 users, latencies stay below 350ms i.e, below threshold.

5.2 Consistency Verification

Consistency testing verified that all clients converge to identical states despite concurrent modifications and network variability.

Table 2 . Consistency Measuring Table

Test Run	2User Count	Total Edits	Hash Matches
1	3	487	3/3
2	5	823	5/5
3	3(packet loss)	512	3/3
4	10	1247	10/10

5.3 Resource Utilization

Resource monitoring assessed the computational overhead of real-time collaboration on both client and server.

Table 3 Client-Side Resource Utilization

User Count	CPU (%)	Memory (MB)	Network (KB/s)
2	10–15	150	10–20
5	25–40	180	40–70
10	45–60	220	120–180
20 (stress)	75–85	280	350–450

Table 4 Server-Side Resource Utilization

Metric	Idle	Light Activity	Heavy Activity
CPU Usage (%)	3–5	8–15	20–35
Memory (MB)	120–150	180–220	280–350
Network RX (KB/s)	1–2	5–12	25–45
Network TX (KB/s)	0.5–1	3–8	15–30

5.4 Feature Comparison

We compared Unified Code with three popular collaborative coding platforms: Replit, CodePen, and VS Code Share. Unified Code offers a unique feature set, especially with the combination of whiteboard, extensive customizability, and being free. While platforms like Replit provide more feature complete IDE features-including file systems and version control-Unified Code focuses specifically on the collaborative coding session experience, making it simpler and more accessible for its target use cases.

Table 5 Feature Comparison of Collaborative Coding Platforms

Feature	UnifiedCode	Replit	CodePen	VS Code Live Share
Real-Time Collaboration	✓	✓	Limited	✓
Integrated Chat	✓	✓	×	×
Whiteboard	✓	×	×	×
Multi-Language Execution	✓ (40+)	✓ (50+)	Limited (3)	×
Browser-Only (No Install)	✓	✓	✓	×
Multi-File Support	✓	✓	Limited	✓
Theme Customization	✓	Limited	✓	✓
Free Tier	✓	Limited	Limited	Requires VS Code

6 Conclusion And Future Work

UnifiedCode provides services for remote development teams for pair programming and code review, recruiters conducting live coding interviews, instructors who demonstrate concepts that students can follow along with immediately, workshop facilitators sharing live coding sessions, and developers mentoring newcomers through interactive walkthroughs. Current limitations include the scalability ceiling at 20 concurrent users, ephemeral sessions without persistence, no authentication system, required continuous connectivity, occasional latency spikes during high-frequency drawing, and single-file execution, which prevents multi-file projects.

Other future directions include hybrid concurrency control for better scalability, persistence and version control to save sessions and file history, authentication for role-based access control, voice and video integration for communication, AI-powered code assistance, support for multi-file projects with package management. Its success

helps to confirm that tool fragmentation imposes considerable cognitive costs and that theoretically suboptimal solutions can deliver fantastic practical performance when matched up to the appropriate use case. Thanks to modern web technologies, production-grade development tools can now be created entirely in the browser. UnifiedCode provides a focused platform for comprehensive collaboration accessible to anyone with an internet connection, with an overall user satisfaction rating of 4.6/5.

Acknowledgements. We would like to extend our sincere thanks to Preeti Garg for the valuable guidance, mentorship, and encouragement throughout this research project. We all thank KIET Group of Institutions for all the computational resources and infrastructure.

We thank all the users in the user study who took the time to try UnifiedCode and give us extensive feedback, which was fundamental to shaping the platform.

Lastly, we would like to express our gratitude to our peers and colleagues who participated in unofficial testing sessions, finding bugs and proposing feature enhancements that helped UnifiedCode become more dependable and usable.

References

- [1] Fraser, N.: Differential Synchronization. Online: <https://neil.fraser.name/writing/sync/> (2009)
- [2] Sun, C., Jia, X., Zhang, Y., Yang, Y., Chen, D.: Achieving convergence, causality preservation, and intention preservation in real-time cooperative editing systems. *ACM Transactions on Computer-Human Interaction* **5**(1), 63–108 (1998)
- [3] Shapiro, M., Preguiça, N., Baquero, C., Zawirski, M.: Conflict-free replicated data types. In: *Proceedings of the 13th International Conference on Stabilization, Safety, and Security of Distributed Systems*, pp. 386–400 (2011)
- [4] Kleppmann, M., Wiggins, A.: Crdts: Consistency without concurrency control. *Communications of the ACM* **62**(6), 48–55 (2019)
- [5] Haverbeke, M.: CodeMirror: In-browser Code Editor. Online: <https://codemirror.net> (2020)
- [6] Microsoft: Visual Studio Live Share. Online: <https://visualstudio.microsoft.com/services/live-share/> (2025)
- [7] Replit Team: Replit: Build Software Faster. Online: <https://replit.com> (2025)
- [8] Fette, I., Melnikov, A.: The WebSocket Protocol. RFC 6455 (2011)
- [9] Rauch, G.: Socket.IO: Realtime Application Framework. Online: <https://socket.io> (2010)
- [10] Gaspar, H.: Judge0: Open-source Online Code Execution System. Online: <https://judge0.com> (2018)

- [11] Engineer Man: Piston: High Performance Code Execution Engine. Online: <https://github.com/engineer-man/piston> (2020)
- [12] Petrosyan, S.: tldraw: A Tiny Little Drawing App. Online: <https://tldraw.com> (2021)
- [13] Miller, R.B.: Response time in man-computer conversational transactions. In: Proceedings of the AFIPS Fall Joint Computer Conference, pp. 267–277 (1968)
- [14] Nielsen, J.: Usability Engineering. Academic Press, ??? (1993)
- [15] Jahns, K.: Yjs: A CRDT Framework for Collaborative Applications. Online: <https://yjs.dev> (2020)
- [16] Lamport, L.: Time, clocks, and the ordering of events in a distributed system. Communications of the ACM **21**(7), 558–565 (1978)
- [17] Sun, C., Ellis, C.: Operational transformation in real-time group editors: Issues, algorithms, and achievements. In: Proceedings of the ACM Conference on Computer Supported Cooperative Work, pp. 59–68 (1998)
- [18] Bailis, P., Ghodsi, A.: Eventual consistency today: Limitations, extensions, and beyond. Communications of the ACM **56**(5), 55–63 (2013)
[\[1–18\]](#)